

ETSI EN 319 421 TSA Policy and Security Requirements

Goya Ledger Time-Stamping Authority

Field	Value
Document identifier	GOYA-TSA-POL-001
Version	1.0
Classification	Public
TSA Policy OID	1.3.6.1.4.1.99999.1.1 (placeholder — IANA PEN registration pending; OID will be updated upon assignment)
Applicable standard	ETSI EN 319 421 V1.1.1 (2016-03)
Supplementary standards	ETSI EN 319 422, RFC 3161, RFC 5816
Effective date	2026-08-13
Review cycle	Annual, or upon material change

1. Scope

This document defines the policy and security requirements for the Goya Ledger Time-Stamping Authority (hereinafter “the TSA”). It is structured in accordance with ETSI EN 319 421 “Policy and Security Requirements for Trust Service Providers issuing Time-Stamps” and addresses the obligations, practices, and technical controls that govern the issuance, management, and verification of time-stamp tokens.

The TSA issues time-stamp tokens that cryptographically bind a hash representation of data to a particular point in time, providing evidence that the data existed at that time. Tokens are produced in two formats:

- **JSON** – structured `TimeStampToken` for application-level consumption.
- **DER** – binary RFC 3161 `TimeStampResp` encapsulating a `TSTInfo` structure within a CMS `SignedData` envelope, per RFC 3161 Section 2.4.2.

The TSA is implemented in the module `src/tsa/mod.rs` with DER encoding logic in `src/tsa/rfc3161_der.rs`. Time acquisition is governed by the pluggable `TimeSource` trait defined in `src/time_source.rs`.

This policy applies to all parties that rely on time-stamp tokens issued under OID 1.3.6.1.4.1.99999.1.1.

2. References

2.1 Normative references

Reference	Title
ETSI EN 319 421	Policy and Security Requirements for Trust Service Providers issuing Time- Stamps
ETSI EN 319 422	Time-stamping protocol and time- stamp token profiles
RFC 3161	Internet X.509 Public Key Infrastructure Time-Stamp Protocol (TSP)
RFC 5652	Cryptographic Message Syntax (CMS)
RFC 5816	ESSCertIDv2 Update for RFC 3161
FIPS 180-4	Secure Hash Standard (SHA-256)
FIPS 202	SHA-3 Standard (SHA3-256)
FIPS 186-5	Digital Signature Standard (Ed25519)
FIPS 204	Module-Lattice-Based Digital Signature Standard (ML-DSA-65)
ETSI TS 101 733	CMS Advanced Electronic Signatures (CAAdES)

2.2 Informative references

Reference	Title
ETSI EN 319 401	General Policy Requirements for Trust Service Providers
ETSI TS 119 312	Cryptographic Suites for Electronic Signatures and Seals
ISO/IEC 27001	Information security management systems
Chile Ley 19.799	Ley sobre Documentos Electronicos, Firma Electronica y Servicios de Certificacion
EU eIDAS Regulation	

Reference	Title
	Regulation (EU) No 910/2014 on electronic identification and trust services

3. Definitions, abbreviations, and notation

3.1 Definitions

Time-stamp token (TST): A data structure issued by the TSA that cryptographically binds a hash representation of a datum to a particular time, as defined in RFC 3161.

Time-stamp request: A request message submitted by a requester to the TSA, containing a message imprint (hash) and optional parameters including nonce and ordering preference.

Time-stamp response: A response message returned by the TSA containing a status indication and, upon success, a time-stamp token.

Message imprint: A hash of the data to be time-stamped, computed by the requester using an approved hash algorithm prior to submission.

Generation time: The time at which the TSA created the time-stamp token, expressed in UTC.

Serial number: A monotonically increasing integer uniquely identifying each time-stamp token issued by the TSA.

Time-Stamping Unit (TSU): The set of hardware and software managed as a unit that creates and signs time-stamp tokens. In the Goya Ledger architecture, this corresponds to a single `TsaProvider` instance (see `src/tsa/mod.rs`, line 99).

Trusted time source: A time source whose accuracy and integrity have been validated within specified tolerances, implemented via the `TimeSource` trait (`src/time_source.rs`, line 51).

3.2 Abbreviations

Abbreviation	Expansion
CAdES	CMS Advanced Electronic Signatures
CMS	Cryptographic Message Syntax
DER	Distinguished Encoding Rules
DID	Decentralized Identifier
ML-DSA	Module-Lattice-Based Digital Signature Algorithm
NTP	Network Time Protocol
OID	Object Identifier

Abbreviation	Expansion
PQC	Post-Quantum Cryptography
TSA	Time-Stamping Authority
TST	Time-Stamp Token
TSU	Time-Stamping Unit
UTC	Coordinated Universal Time

4. General concepts

4.1 Time-stamping service

The Goya Ledger TSA provides a time-stamping service that enables subscribers to obtain cryptographic proof that specific data existed at a given point in time. The service operates as an integral component of the Goya Ledger blockchain node, identified by a DID of the form `did:goya:{pubkey_hex[. .16]}`.

The TSA accepts time-stamp requests containing a message imprint (a 256-bit hash digest, hex-encoded to 64 characters) and returns a signed time-stamp token that includes:

- The message imprint echoed from the request.
- The generation time in UNIX seconds (UTC).
- A monotonically increasing serial number.
- The TSA policy OID.
- Accuracy indication.
- An optional nonce for replay protection.
- The cryptographic signature of the TSA.

4.2 Time-stamp token formats

4.2.1 JSON format

The `issue()` method of `TsaProvider` (`src/tsa/mod.rs`, line 194) returns a `TimeStampResponse` containing a `TimeStampToken` with a nested `TstInfo` structure. The signature is computed over a deterministic canonical string representation produced by `TstInfo::signing_payload()` (line 307).

4.2.2 DER format

The `issue_der()` method (`src/tsa/mod.rs`, line 264) returns a binary DER-encoded `TimeStampResp` per RFC 3161 Section 2.4.2. The response comprises:

1. `PKIStatusInfo` – status code (0 = granted).
2. `ContentInfo` – CMS SignedData (version 3) wrapping a DER-encoded `TSTInfo`.

The DER encoding logic resides in `src/tsa/rfc3161_der.rs`. The `SignedData` structure includes signed attributes (`contentType`, `messageDigest`, `signingTime`) per RFC 5652 Section 5.4.

4.2.3 CAdES-T integration

Time-stamp tokens in DER format may be embedded as unsigned attributes in CAdES-BES signatures to produce CAdES-T signatures, as implemented in `src/signature/cades_der.rs`. The TSA token DER is carried in the `unsignedAttrs` field of `SignerInfo`.

4.3 Trust model

Relying parties trust the TSA on the basis that:

- The TSA signing key is protected and used exclusively for time-stamping.
 - The TSA's clock is synchronised to UTC within stated accuracy.
 - Serial numbers are unique and monotonically increasing.
 - The TSA validates its own operational integrity before each issuance.
-

5. TSA obligations and practices

5.1 TSA obligations

The TSA shall:

1. **Use a trustworthy time source.** The TSA shall only issue time-stamp tokens when the configured time source passes validation. If `time_source.validate()` fails, the TSA shall reject the request with status 2 ("time source not trusted"). See `validated_time()` in `src/tsa/mod.rs`, line 183.
2. **Include a trustworthy time value.** Each time-stamp token shall contain a `genTime` value obtained from the validated time source, expressed in UTC.
3. **Include a unique serial number.** Each token shall contain a serial number sourced from an `AtomicU64` counter that is monotonically increasing and never reused. Serial numbers are persisted to disk via `persist_serial()` (line 165) and reloaded on restart via `with_serial_path()` (line 128) to guarantee they never go backwards, even across process restarts.
4. **Sign each token using an approved algorithm.** Tokens are signed using the node's `SigningProvider`, supporting:
 - Ed25519 (FIPS 186-5) – 64-byte signatures.
 - ML-DSA-65 (FIPS 204) – 3309-byte post-quantum signatures.
 - RSA with SHA-256 (PKCS#1 v1.5) – variable-length signatures.

5. **Validate operational health before issuance.** The TSA performs a self-check of the signing subsystem before every issuance by signing and verifying a test message (`validate_signer()`, line 171). If self-verification fails, the request is rejected.
6. **Validate message imprints.** The TSA shall reject requests whose message imprint is not exactly 64 hexadecimal characters or contains invalid hex (`validate_imprint()`, line 334).
7. **Echo the nonce.** When a request contains a nonce, the TSA shall include the identical nonce value in the response token for replay protection.
8. **Publish this policy.** The TSA shall make this policy document available to subscribers and relying parties. The policy OID is returned in every token and via the `policy_info()` method (line 293).

5.2 Subscriber obligations

Subscribers (requesters of time-stamp tokens) shall:

1. Compute the message imprint using an approved hash algorithm (SHA-256 or SHA3-256) before submitting the request.
2. Verify the time-stamp token signature upon receipt.
3. Verify that the message imprint in the token matches the submitted value.
4. Include a nonce in the request when replay protection is required.
5. Check the TSA policy OID in the response matches the expected value.

5.3 Relying party obligations

Relying parties shall:

1. Verify the time-stamp token signature using the TSA's public key.
2. Confirm the validity of the signing algorithm at the time of reliance.
3. Verify the token's policy OID.
4. Check the accuracy value to determine the time window within which the datum is proven to have existed.
5. Verify the nonce if one was included in the original request.

Token verification is provided by `verify_token()` (`src/tsa/mod.rs`, line 324) for JSON tokens and `verify_timestamp_resp_der()` (`src/tsa/rfc3161_der.rs`, line 347) for DER tokens.

6. TSA policy requirements

6.1 Key management

6.1.1 TSA key generation

The TSA signing key pair is generated through the `SigningProvider` abstraction (`src/identity/signing/`). Key generation uses:

- **Ed25519**: `SoftwareSigningProvider::generate()` – deterministic key derivation from cryptographically secure random seed.
- **ML-DSA-65**: `MlDsaSigningProvider::generate()` – FIPS 204 compliant key generation.
- **RSA**: `RsaSigningProvider::generate()` – RSA key pair generation with SHA-256 digest.

All key generation depends on the `ppc_crypto_module` crate (`crates/ppc_crypto_module/`), which serves as the exclusive cryptographic boundary. Direct imports of `sha2`, `ed25519_dalek`, or other primitive crates outside this boundary are prohibited and enforced by the `crypto_boundary` integration test.

6.1.2 TSA key protection

The TSA signing key shall be:

- Used exclusively for time-stamping operations.
- Protected against unauthorised access through the `SigningProvider` trait encapsulation.
- Never exported or serialised to logs.
- Stored in memory within an `Arc<dyn SigningProvider>` with no public accessor for the private key material.
- Zeroized on process termination via the `zeroize` trait.

Current status: Software-only key storage (in-process memory). This does not satisfy ETSI EN 319 421 Section 7.2.2 or ETSI EN 319 401 requirements for secure cryptographic device protection (FIPS 140-2 Level 2+ or equivalent). HSM integration is planned for 2027-Q1 per `docs/compliance/EU-COMPLIANCE-GAPS.md` Gap 10. Until HSM is deployed, the TSA operates at non-qualified assurance level.

The public key is available via `SigningProvider::public_key()` and is hex-encoded in each token for verification purposes.

6.1.3 TSA key usage period

The TSA signing key shall be replaced at intervals not exceeding 2 years (crypto-period), or immediately upon suspected compromise. Key lifecycle management follows PS06 (Key Management Plan, `docs/compliance/PS06-KEY-MANAGEMENT-PLAN.md`). When a key is replaced:

- Existing tokens remain verifiable using the original public key.
- New tokens are signed with the replacement key.

- The serial number counter continues from its current value; it is never reset.

6.1.4 End of key life cycle

When a TSA signing key reaches end of life:

- The key shall no longer be used for signing new tokens.
- The public key shall remain available for verification of previously issued tokens for 7 years post-retirement.
- The serial number file shall be preserved to prevent serial number reuse.
- The TSA certificate associated with the retired key shall be revoked via CRL and OCSP status update. Relying parties can verify key status at <https://goya.cl/pki/ocsp> and <https://goya.cl/pki/crl.der>.

6.1.5 Cryptographic algorithm requirements

The TSA supports the following algorithm combinations:

Signature algorithm	OID	Hash algorithm	Hash OID
Ed25519	1.3.101.112	SHA-256	2.16.840.1.101.3.4.2.1
Ed25519	1.3.101.112	SHA3-256	2.16.840.1.101.3.4.2.8
ML-DSA-65	2.16.840.1.101.3.4.3.17	SHA-256	2.16.840.1.101.3.4.2.1
ML-DSA-65	2.16.840.1.101.3.4.3.17	SHA3-256	2.16.840.1.101.3.4.2.8
RSA (SHA-256)	1.2.840.113549.1.1.11	SHA-256	2.16.840.1.101.3.4.2.1
RSA (SHA-256)	1.2.840.113549.1.1.11	SHA3-256	2.16.840.1.101.3.4.2.8

Algorithm selection is controlled by the `SIGNING_ALGORITHM` environment variable and the `SigningAlgorithm` enum (`src/identity/signing/`).

The inclusion of ML-DSA-65 provides post-quantum cryptographic readiness in accordance with NIST FIPS 204 and ETSI recommendations for quantum-safe migration.

Cryptographic Algorithm Validation Program (CAVP) test vectors are maintained for SHA-256, SHA3-256, and Ed25519 to ensure implementation correctness.

6.2 Time-stamping

6.2.1 Time-stamp token profile

Each time-stamp token conforms to the `TSTInfo` structure defined in RFC 3161 Section 2.4.2:

Field	ASN.1 type	Value	Reference
version	INTEGER	1	RFC 3161 Section 2.4.2
policy		1.3.6.1.4.1.99999.1.1	

Field	ASN.1 type	Value	Reference
	OBJECT IDENTIFIER		TSA_POLICY_OID constant
messageImprint	MessageImprint	Hash algorithm OID + OCTET STRING	Request echo
serialNumber	INTEGER	Monotonically increasing u64	AtomicU64 counter
genTime	GeneralizedTime	UTC timestamp	Validated time source
accuracy	Accuracy	1 second (default)	DEFAULT_ACCURACY_SECS
ordering	BOOLEAN	As requested	Request echo
nonce	INTEGER	As requested (optional)	Request echo
tsa	GeneralName	directoryName with commonName = TSA DID	TSA identity

6.2.2 Request processing

Upon receipt of a `TimeStampRequest`, the TSA performs the following steps in order:

1. **Signer health check.** Call `validate_signer()`. If the signing subsystem is unhealthy, reject with status 2.
2. **Imprint validation.** Call `validate_imprint()`. The imprint must be exactly 64 hexadecimal characters (256-bit hash). Reject if invalid.
3. **Time acquisition.** Call `validated_time()`. If a `TimeSource` is configured, invoke `validate()` first; reject if validation fails. Obtain UNIX seconds from the source.
4. **Serial assignment.** Atomically increment the serial counter via `fetch_add(1, SeqCst)`. Persist the new value to disk.
5. **TSTInfo construction.** Assemble the `TstInfo` (JSON) or DER-encoded `TSTInfo` structure.
6. **Signing.** Sign the canonical payload (JSON) or the DER signed attributes (DER format). Return the signed token.

6.2.3 Response status codes

Status	Meaning	Condition
0	Granted	Request processed successfully
2	Rejection	Signer unhealthy, invalid imprint, time source untrusted, or signing failure

6.2.4 Serial number management

Serial numbers are managed by an AtomicU64 counter (`src/tsa/mod.rs`, line 101) with the following guarantees:

- **Uniqueness.** `fetch_add(1, SeqCst)` ensures each serial is assigned exactly once, even under concurrent access.
- **Monotonicity.** Serial numbers strictly increase. The counter is never decremented or reset.
- **Persistence.** When `serial_path` is configured, the counter is written to disk after each issuance (`persist_serial()`, line 165).
- **Restart safety.** On startup, `with_serial_path()` loads the persisted value and takes `max(saved_serial, epoch_seed)` to ensure the counter never goes backwards, even if the persisted file is stale or missing (line 136).
- **Initial seeding.** When no persisted value exists, the counter is seeded from `SystemTime::now()` as UNIX seconds, providing a high starting value that avoids overlap with prior runs.

6.2.5 Accuracy

The default accuracy is 1 second (`DEFAULT_ACCURACY_SECS = 1`, line 27). This value represents the maximum deviation between the reported `genTime` and UTC. The accuracy may be configured per TSA instance via `with_accuracy()` (line 144).

The accuracy field is included in every token, enabling relying parties to compute the time window [`genTime - accuracy`, `genTime + accuracy`] within which the datum is proven to have existed.

6.3 Clock synchronisation

6.3.1 Time source architecture

The TSA obtains time through the `TimeSource` trait (`src/time_source.rs`, line 51), which defines:

- `now()` – returns a `TrustedTimestamp` containing UNIX seconds, source type, and accuracy.
- `source_type()` – identifies the source (System, NTP, or Simulated).
- `validate()` – confirms the source is operating within acceptable parameters.

Three implementations are provided:

Implementation	Module	Use
System clock	<code>src/time_source.rs</code>	Default, no NTP validation
NTP-synchronised clock	<code>NtpTimeSource</code>	Production, validates sync status
Simulated clock	<code>SimulatedTimeSource</code>	Testing only

6.3.2 NTP enforcement

When the TSA is configured with an `NtpTimeSource`:

- The TSA calls `validate()` before every issuance.
- `validate()` checks that the NTP client reports synchronised status and that drift does not exceed the configured maximum (`DEFAULT_MAX_DRIFT_SECS = 5`, defined in `src/time_source.rs`, line 15).
- If validation fails, the TSA rejects the request with a `TimeError::NtpNotSynced` or `TimeError::DriftExceeded` error, and no token is issued.
- This behaviour is verified by tests `rejects_when_ntp_desynced` and `rejects_der_when_ntp_desynced` in `src/tsa/mod.rs`.

6.3.3 Clock drift tolerance

The maximum acceptable clock drift from the NTP reference is 5 seconds by default. This tolerance is configurable and shall be set no higher than the TSA's stated accuracy to ensure that the accuracy claim in issued tokens is truthful.

6.3.4 Time source audit trail

Each `TrustedTimestamp` includes a `source` field indicating which time source type was used (`System`, `Ntp`, or `Simulated`) and an `accuracy_secs` field. These values shall be recorded in the audit log alongside each time-stamp issuance event.

6.4 Audit and logging

6.4.1 Audit log requirements

The TSA shall maintain an audit log recording at a minimum:

- Every time-stamp request received (hash algorithm, imprint, nonce, ordering flag).
- Every time-stamp response issued (serial number, generation time, status code).
- Every rejection, including the reason for rejection.
- The time source type and validation status for each issuance.
- All administrative actions affecting the TSA (key rotation, configuration changes).

6.4.2 Audit log integrity

When the node operates with `STORAGE_BACKEND=rocksdb`, audit logs are persisted to the RocksDB store. Audit log entries shall be:

- Append-only; no entry shall be modified or deleted during the retention period.
- Integrity-protected via hash chain: each log entry includes the SHA3-256 hash of the previous entry, forming a verifiable chain. Chain integrity is validated continuously (on each append) and via daily batch verification (`verify_audit_chain()`).
- Protected by the same access controls as the TSA signing key.
- Available for review by authorised auditors upon request.

- Long-term archival: logs older than 1 year are exported to signed archive files (ML-DSA-65 signature over the archive hash) for offline retention, ensuring integrity verification survives RocksDB compaction or format changes.

6.4.3 Audit log retention

Audit logs shall be retained for a minimum of seven (7) years, or longer if required by applicable regulation (e.g., Chile Ley 19.799, EU eIDAS).

6.4.4 Audit frequency

An internal audit of the TSA operations shall be conducted:

- At least annually.
- Following any security incident.
- Following any material change to the TSA software, key material, or time source configuration.
- Following any change to the algorithms listed in Section 6.1.5.

6.5 Token verification

6.5.1 JSON token verification

The `verify_token()` function (`src/tsa/mod.rs`, line 324) verifies a JSON time-stamp token by:

1. Recomputing the deterministic signing payload from the `TstInfo` fields.
2. Verifying the signature against the payload using the public key and algorithm declared in the token.
3. Dispatching to the shared `verify_signature()` function in `src/signature/`.

6.5.2 DER token verification

The `verify_timestamp_resp_der()` function (`src/tsa/rfc3161_der.rs`, line 347) verifies a DER time-stamp response by:

1. Parsing the `TimeStampResp` outer SEQUENCE and checking `PKIStatus == 0`.
 2. Extracting the CMS SignedData from the `ContentInfo`.
 3. Locating the `encapContentInfo` with OID `id-ct-TSTInfo` (1.2.840.113549.1.9.16.1.4).
 4. Extracting the DER-encoded `TSTInfo` from the OCTET STRING.
 5. Computing SHA-256 over the `TSTInfo` bytes and comparing with the `messageDigest` signed attribute.
 6. Reconstructing the SET-tagged signed attributes and verifying the cryptographic signature.
 7. Returning the parsed `TstInfoFields` (serial number, generation time, message imprint, policy OID, nonce, ordering).
-

7. Security management

7.1 Security policy

The TSA is operated in accordance with the information security policies of the Goya Ledger project, which address:

- Physical security of the systems hosting the TSA.
- Logical access controls, including the ACL_MODE enforcement mechanism.
- Network security, including TLS requirements for production (RUST_BC_ENV=production mandates TLS_CERT_PATH and TLS_KEY_PATH).
- Personnel security and separation of duties.
- Incident response and disaster recovery.

7.2 Risk assessment

A risk assessment addressing the TSA shall be performed:

- Before initial deployment.
- Annually thereafter.
- Upon material changes to the system architecture, threat landscape, or regulatory environment.

The risk assessment shall consider at minimum:

- Compromise of the TSA signing key.
- Compromise or failure of the time source.
- Denial of service against the TSA.
- Serial number exhaustion or rollback.
- Algorithm obsolescence (including quantum threat to pre-quantum algorithms).

7.3 Asset classification

The following assets are classified as critical:

Asset	Classification	Protection measure
TSA signing private key	Confidential	In-memory only, no export API, Arc<dyn SigningProvider> encapsulation
Serial number counter	Integrity-critical	AtomicU64 with SeqCst ordering, disk persistence, max-on-reload
Time source	Integrity-critical	validate() gate before every issuance, NTP drift enforcement
Audit log	Integrity-critical	Append-only, RocksDB persistence, retention policy

Asset	Classification	Protection measure
TSA policy OID	Public	Constant TSA_POLICY_OID in source code

7.4 Personnel security

All personnel with administrative access to systems hosting the TSA shall:

- Be subject to background verification at hiring and re-screened every 3 years.
- Hold a defined trusted role (TSA Administrator, Security Officer, or PKI Operator) per PE01 (docs/compliance/PE01-PERSONNEL-EVALUATION.md).
- Receive initial training (minimum 4 hours) on this policy, PS06 key management procedures, and incident response per PS07.
- Receive annual refresher training covering policy updates and operational lessons learned.
- Sign the Acceptable Use Policy (GOYA-AUP-001, docs/policy/ACCEPTABLE-USE-POLICY.md).
- Acknowledge their responsibilities under this policy.
- Be subject to the disciplinary process defined in GOYA-PS02-001 section 6.2.3.

Dual-control (two-person authorization) is required for TSA key generation, key destruction, and policy changes.

7.5 Physical security

Systems hosting the TSA are deployed on Fly.io infrastructure (region IAD) with provider-managed physical controls verified via SOC 2 Type II reports. The provider's data center certifications are reviewed annually by the Security Officer. Specific controls include:

- Controlled access to server rooms with biometric + badge readers (provider-managed).
- Environmental controls: fire suppression (FM-200), HVAC, redundant power with UPS and diesel generator (provider-managed).
- Physical tamper detection and response for any hardware security modules in use.
- Physical security policy documented in SF01 (docs/compliance/SF01-PHYSICAL-SECURITY.md).

7.6 Operations management

7.6.1 Change management

Changes to the TSA software, configuration, or infrastructure shall follow the project's change management process, including:

- Peer review of code changes.
- Pre-commit quality gate: `cargo fmt --check`, `cargo clippy -- -D warnings`, `cargo test --lib`.
- Crypto boundary enforcement via the `crypto_boundary` integration test.

- Regression testing of all TSA tests (unit tests in `src/tsa/mod.rs` and DER tests in `src/tsa/rfc3161_der.rs`).

7.6.2 Capacity and performance

The TSA shall be monitored for:

- Request throughput and latency.
- Serial number counter headroom (the u64 counter supports over 1.8×10^{19} values).
- Disk space for serial persistence and audit logs.
- Time source availability and drift.

7.6.3 Incident management

Security incidents affecting the TSA shall be:

- Detected through monitoring and audit log analysis.
- Reported to the designated security contact.
- Investigated, contained, and remediated.
- Documented in an incident report, including root cause and corrective actions.
- Reported to relevant supervisory bodies where required by applicable law.

7.7 Access control

Access to TSA administration functions shall be restricted to authorised personnel. The `ACL_MODE` configuration controls API-level access enforcement. In production:

- `ACL_MODE=permissive` generates a warning.
- All API endpoints are protected by `enforce_acl` (see `src/api/`).
- Rate limiting is enforced via `RATE_LIMIT_RPS`, `RATE_LIMIT_RPM`, and `RATE_LIMIT_RPH`.

7.8 Network security

- All production traffic shall be encrypted using TLS with certificates specified by `TLS_CERT_PATH` and `TLS_KEY_PATH`.
- CORS origins are restricted via `CORS_ALLOWED_ORIGINS`.
- P2P network traffic uses TCP/TLS (see `src/network/`).

7.9 Cryptographic module

All cryptographic operations are channelled through the `pqc_crypto_module` crate (`crates/pqc_crypto_module/`). This boundary is enforced by the `crypto_boundary` integration test, which verifies that no source file outside the crate directly imports primitive cryptographic libraries.

7.10 Business continuity

The TSA shall implement business continuity measures including:

- Serial number persistence to disk, enabling recovery after process restart without serial rollback.
- Configurable time source with graceful rejection when NTP is unavailable (rather than silent degradation).
- Structured logging (LOG_FORMAT=json) for automated monitoring and alerting.
- Deployment via Docker Compose with multi-node support for high availability.

Recovery objectives:

Metric	Target
Recovery Time Objective (RTO)	4 hours
Recovery Point Objective (RPO)	0 (serial number persisted on each issuance)
Maximum tolerable downtime	24 hours

7.11 Termination

Upon termination of the TSA service, the following procedure shall be executed with a minimum notice period of 90 days:

1. Publish termination notice on <https://goya.cl/pki/> and notify all known relying parties via email.
2. Cease issuance of new tokens on the termination date.
3. The TSA signing key shall be securely destroyed in a witnessed ceremony per PS06.
4. The serial number file shall be archived.
5. All audit logs shall be preserved for the full 7-year retention period and transferred to a successor entity or the supervisory authority.
6. The public key shall remain available for verification of previously issued tokens for 7 years post-termination.
7. Existing tokens remain verifiable using the archived public key.

Annex A: TSA self-check procedure (informative)

Before every token issuance, the TSA executes `validate_signer()` (`src/tsa/mod.rs`, line 171):

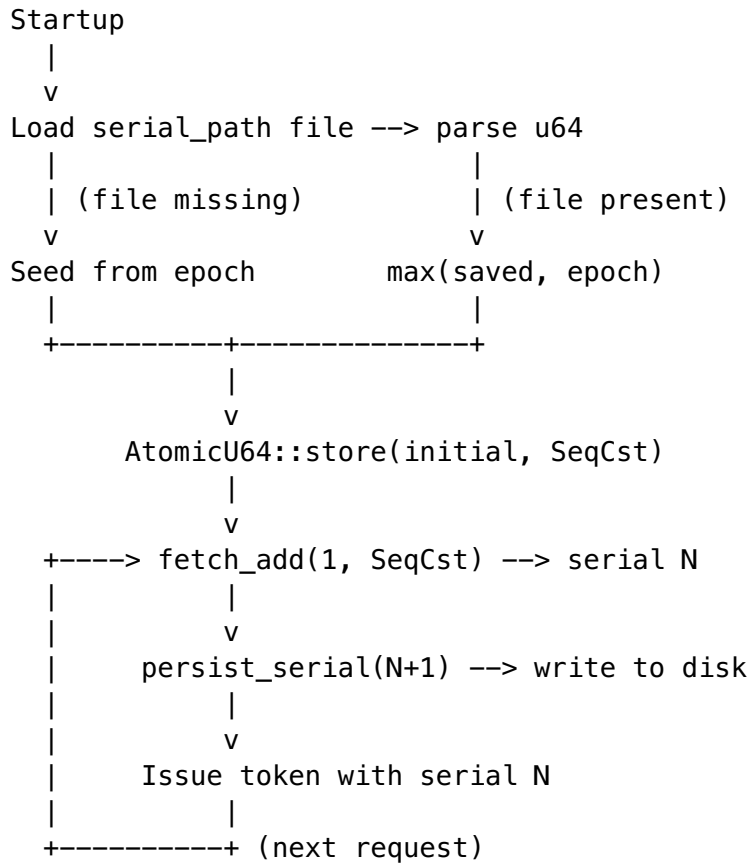
1. Generate test message: `b"tsa-self-check"`
2. Sign the test message with the TSA signing key.
3. Verify the signature against the test message using the TSA public key.
4. If verification fails, reject the pending request (status 2).

This procedure ensures that:

- The signing key is available and functional.

- The signing and verification paths are consistent.
- Hardware or software faults in the cryptographic subsystem are detected before issuing a token that would be unverifiable.

Annex B: Serial number lifecycle (informative)



This design guarantees:

- Serials never repeat, even across restarts.
- Serials never decrease, even if the persisted file is corrupted or stale.
- The SeqCst memory ordering prevents reordering under concurrent access.

Annex C: Approved hash algorithms (informative)

Algorithm	OID	Output size	Standard
SHA-256	2.16.840.1.101.3.4.2.1	256 bits	FIPS 180-4
SHA3-256	2.16.840.1.101.3.4.2.8	256 bits	FIPS 202

Both algorithms are implemented in `crates/pqc_crypto_module/` via the `hash_with()` function and are validated against CAVP test vectors.

Annex D: DER structure reference (informative)

The DER-encoded TimeStampResp produced by `build_timestamp_resp_der()` (`src/tsa/rfc3161_der.rs`, line 288) has the following ASN.1 structure:

```
TimeStampResp ::= SEQUENCE {
    status          PKIStatusInfo,          -- SEQUENCE { status
        INTEGER (0) }
    timeStampToken  ContentInfo             -- CMS SignedData
}

ContentInfo ::= SEQUENCE {
    contentType     OBJECT IDENTIFIER,      -- id-signedData
        (1.2.840.113549.1.7.2)
    content         [0] EXPLICIT SignedData
}

SignedData ::= SEQUENCE {
    version          INTEGER (3),
    digestAlgorithms SET OF AlgorithmIdentifier,
    encapContentInfo EncapsulatedContentInfo,
    signerInfos      SET OF SignerInfo
}

EncapsulatedContentInfo ::= SEQUENCE {
    eContentType     OBJECT IDENTIFIER,      -- id-ct-TSTInfo
        (1.2.840.113549.1.9.16.1.4)
    eContent         [0] EXPLICIT OCTET STRING CONTAINING TSTInfo
}

SignerInfo ::= SEQUENCE {
    version          INTEGER (3),
    sid              SubjectKeyIdentifier,
    digestAlgorithm   AlgorithmIdentifier,
    signedAttrs      [0] IMPLICIT SET OF Attribute,
    signatureAlgorithm AlgorithmIdentifier,
    signature         OCTET STRING
}
```

Interoperability with third-party ASN.1 parsers is validated by integration tests using the `x509-parser` crate (see tests in `src/tsa/rfc3161_der.rs`).

End of document.